

SQL + MongoDB — Side by Side

Backend Engineer ke liye Database Course • PDF 3 of 5 • Parts 9–12

Part	Topic	Kyun important
9	Data Modeling — Normalization vs Embed/Reference	System design interview ka pehla sawaal
10	Indexes — single, compound (ESR), unique, TTL, partial	80% performance problems ka solution
11	EXPLAIN — slow query debug karna	Production debugging skill
12	Transactions + ACID — Place Order	Money + inventory ka data kabhi galat nahi hona chahiye

Data wahi hai (PDF 1 section 2.0 + Anita). Jahan naya data chahiye, wahan us section mein diya hai. Ye PDF '**kaise likhte hain**' se zyada '**kyun aur kab**' ke baare mein hai — yahi senior aur junior engineer ka farak hai.

PART 9 — Data Modeling

9.1 Problem: sab kuch ek table mein daal diya

► CONCEPT

Naya developer aksar ek badi 'Excel sheet' jaisi table banata hai. Dekhte hain kya problems aati hain — **normalization inhi problems ka solution hai.**

Output

order_details (ek hi table mein sab)

order_id	user_id	user_name	user_city	product_id	product_name	price	qty	status
01001	1103	Sonu	Mumbai	P101	Steel Pipe	500	3	PAID
01003	1103	Sonu	Mumbai	P102	PVC Pipe	200	1	CANCELLED
01003	1103	Sonu	Mumbai	P103	Elbow Joint	80	1	CANCELLED
01004	1105	Rahul	Delhi	P101	Steel Pipe	500	2	PAID

► TEEN ANOMALIES (INTERVIEW MEIN NAAM SE POOCHHTE HAIN)

Anomaly	Hamare example mein	Result
Update anomaly	Sonu Pune shift hua → 3 rows update karni padengi. Ek bhool gaye to?	Sonu ek jagah Mumbai, ek jagah Pune — data inconsistent
Insert anomaly	Naya product P105 add karna hai, abhi kisi ne order nahi kiya	Row kaise banayein? order_id kya daalein?
Delete anomaly	O1004 delete kiya (Rahul ka akela order)	Rahul ki पूरी info hi gayab ho gayi

Root cause: **ek fact (Sonu ki city) kai jagah likha hai.** Normalization ka simple rule: **har fact ek hi jagah.**

9.2 Normalization — 1NF, 2NF, 3NF (practical version)

► 1NF — HAR CELL MEIN EK VALUE, KOI LIST NAHI

Output

GALAT: orders(order_id, user_id, product_ids)
01003 | 1103 | 'P102,P103' ← ek cell mein list

SAHI: order_items(order_id, product_id, ...)
01003 | P102
01003 | P103

Kyun: **WHERE product_ids LIKE '%P10%'** jaisa hack karna padega, index nahi lagega, FK nahi lagega, P102 ka quantity kahan rakhoge?

► 2NF — COMPOSITE KEY HO TO HAR COLUMN POORI KEY PE DEPEND KARE

Output

```
GALAT: order_items(order_id, product_id, quantity, product_name)
       PK = (order_id, product_id)
       product_name sirf product_id pe depend karta hai, order_id se koi lena-dena nahi
       → har order mein 'Steel Pipe' repeat

SAHI:  order_items(order_id, product_id, quantity, price)
       products(product_id, name, ...)
```

► 3NF — NON-KEY COLUMN DOOSRE NON-KEY COLUMN PE DEPEND NA KARE

Output

```
GALAT: orders(order_id, user_id, user_city, total)
       user_city → user_id pe depend karta hai (order pe nahi)
       order_id → user_id → user_city (transitive dependency)

SAHI:  orders(order_id, user_id, total)
       users(user_id, name, city)
```

Ek line mein yaad rakho

- **1NF:** ek cell = ek value.
- **2NF:** har column पूरी key के बारे में हो, key के तुकड़े के बारे में नहीं.
- **3NF:** har column sirf key के बारे में हो, किसी और column के बारे में नहीं.
- Famous line: *"The key, the whole key, and nothing but the key."*

► FINAL NORMALIZED SQL DESIGN (JO HUM PDF 1 SE USE KAR RAHE HAIN)

SQL (dono)

```
users (user_id PK, name, city, email)
categories (category_id PK, name)
products (product_id PK, name, price, category_id FK, stock)
orders (order_id PK, user_id FK, total, status, created_at)
order_items (order_id FK, product_id FK, quantity, price, PK(order_id, product_id))
payments (payment_id PK, order_id FK UNIQUE, amount, method, status)
```

9.3 Denormalization — jaan-bujhkar duplicate karna

Real production SQL designs **100% normalized nahi** hote. Kabhi-kabhi read speed ya business correctness के लिये jaan-bujhkar data copy करते हैं:

Denormalized column	Kyun	Cost
<code>order_items.price</code>	Business correctness: price काल बदलेगा, पुराना bill नहीं बदलना चाहिये. Ye duplication नहीं, historical snapshot है.	कोई नहीं — ये सही design है
<code>orders.total</code>	Har baar items SUM करने की ज़रूरत नहीं, list page fast	Items बदले तो total भी update (transaction में)
<code>products.review_count</code>	Listing पे COUNT(*) har baar mehenga	Review add/delete पे counter update
<code>orders.shipping_city</code>	Order के वक़्त का address — user बाद में shift हो सकता है	Snapshot, कोई sync नहीं

Rule: पहले normalize करो, फिर **measured** performance problem या business need हो तब denormalize करो — और लिखो कि copy को sync कौन रहेगा.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
3 anomalies 1NF/2NF/3NF ek-ek line mein + example - Snapshot (price) vs duplicate - Denormalize consciously	Cached counters/totals ka sync cost	- BCNF, 4NF, 5NF - Formal functional dependency theory

9.4 MongoDB modeling — Embed vs Reference

► CONCEPT

MongoDB mein pehla sawaal 'normalize kaise karein' nahi hai. Pehla sawaal hai: '**App ye data kaise padhti aur likhti hai?**' (access patterns). Golden rule: **jo data saath padha jaata hai, wo saath rakho.**

	Embed (andar rakho)	Reference (ID rakho)
Kaisa dikhta hai	<code>order.items: [{...}, {...}]</code>	<code>order.userId: "1103"</code>
Read	Ek query mein sab — fast	\$lookup / populate / 2 queries
Write atomicity	Ek document = atomic, transaction nahi chahiye	Multiple docs → transaction chahiye ho sakta
Duplication	Ho sakta (name copy)	Nahi
Size	16 MB limit, document badhta hai	Koi problem nahi
Independent access	Mushkil (item akele query karna awkward)	Easy

► ORDER + ORDERITEMS — DONO DESIGNS

SQL (dono)

```
-- SQL: 3 tables, JOIN se padhna
orders      (order_id, user_id, total, status)
order_items (order_id, product_id, quantity, price)
products    (product_id, name, price, ...)
```

MongoDB

```
// Mongo Option A — EMBED (recommended for order items)
{
  orderId: "01003",
  userId: "1103",                      // reference → user
  status: "CANCELLED",
  total: 280,
  shipping: { city: "Mumbai", pincode: "400069" }, // embed (snapshot)
  items: [                                // embed
    { productId: "P102", name: "PVC Pipe",    quantity: 1, price: 200 },
    { productId: "P103", name: "Elbow Joint",  quantity: 1, price: 80 }
  ],
  createdAt: ISODate("2026-09-05T09:15:00Z")
}

// Mongo Option B — REFERENCE (SQL jaisa, Mongo mein usually galat choice)
orders: { orderId: "01003", userId: "1103", total: 280 }
orderItems: { orderId: "01003", productId: "P102", quantity: 1 }
orderItems: { orderId: "01003", productId: "P103", quantity: 1 }
```

Items embed kyun? (1) Order hamesha items ke saath dikhaya jaata hai. (2) Items limited hote hain (10-50, 10,000 nahi). (3) Items order ke saath hi bante hain, alag se rarely badalte hain. (4) Order create = **ek insert, atomic**, transaction ki zaroorat nahi. (5) Items ka order ke bina koi matlab nahi.

User reference kyun? User independently exist karta hai, uske bahut saare orders honge, user profile alag se padha/badla jaata hai.

► EXTENDED REFERENCE PATTERN — REFERENCE + ZAROORI FIELDS KI COPY

Items mein **productId** (reference) ke saath **name** aur **price** bhi copy kiye. Order page dikhane ke liye products collection padhni hi nahi padi. Ye SQL ke price snapshot jaisa hi hai. Product ka naam badla to? Order mein purana naam hi sahi hai (invoice pe wahi tha).

9.5 Practical rules — kab embed, kab reference

Relationship / situation	Decision	Example
One-to-few (bounded, chhota)	Embed	User → addresses (2-5), Order → items
One-to-many (sau-hazaar), child akele bhi padhe jaate	Reference (child mein parentId) ya parent mein IDs ka array	Product → reviews
One-to-squillions (unbounded)	Reference, child side pe — kabhi parent mein array nahi	User → orders, Product → view logs
Many-to-many	IDs ka array ek/dono taraf, ya alag collection	Product ↔ tags (embed strings)
Data hamesha saath padha jaata hai	Embed	Order + shipping address
Child ko independently query/update karna hai	Reference	Payments (webhooks alag se update karte hain)
Embedded data baar baar badalta hai aur kai jagah copy hai	Reference	User ka naam har comment mein copy — mat karo agar naam badalna common hai
Atomic update chahiye multiple parts pe	Embed (single-doc atomicity)	Cart + cart items

Anti-pattern: Unbounded array

- **users**: { **userId**: "1103", **orders**: [...10,000 orders...] } — document har order ke saath badhta hai, 16 MB cross, har user read pe saare orders load, har update pe bada document rewrite.
- Fix: orders alag collection, **userId** reference, **userId + createdAt** pe index.
- Beech ka raasta — **subset pattern**: product document mein sirf **latest 5 reviews** embed (product page ke liye), saare reviews alag collection mein.

► HAMARA POORA E-COMMERCE — MONGODB DESIGN

MongoDB

```
// users – addresses embed (few, bounded)
users      { userId, name, email, city, addresses: [ { label, line1, pincode } ] }

categories { categoryId, name }

// products – categoryId reference + categoryName (extended ref), flexible specs,
//           rating counters (computed) + latest 5 reviews (subset)
products   { productId, name, price, stock, categoryId, categoryName,
            tags: [String], specs: { diameter, material, ... },
            ratingAvg, reviewCount, recentReviews: [ ...max 5 ] }

// reviews – alag collection (unbounded)
reviews    { productId, userId, rating, text, createdAt }

// orders – shipping snapshot + items embed, user reference
orders     { orderId, userId, status, total, shipping: { ... },
            items: [ { productId, name, price, quantity } ], createdAt }

// payments – alag collection (independent lifecycle: retries, webhooks)
payments   { paymentId, orderId, amount, method, status, gatewayRef, attempts }
```

specs — yahi Mongo ki taakat hai: Steel Pipe ke specs (diameter, thickness) aur Brass Valve ke specs (pressure rating) alag shape ke ho sakte hain. SQL mein iske liye EAV table ya JSONB column chahiye hota.

► PAYMENT EMBED KAREIN YA ALAG?

Dono valid hain. **Alag** chuna kyunki: ek order ke kai payment attempts ho sakte hain (fail → retry), gateway webhooks payment ko independently update karte hain, aur finance team payments ko alag query karti hai. Simple app mein **order.payment: { status, method }** embed bhi theek hai.

9.6 Node.js — schema definitions

```
// ----- Mongoose: embedded subdocuments -----
const orderItemSchema = new mongoose.Schema({
  productId: { type: String, required: true },
  name:      { type: String, required: true },
  price:     { type: Number, required: true, min: 0 },
  quantity:  { type: Number, required: true, min: 1 },
}, { _id: false }); // har item pe alag _id nahi chahiye

const orderSchema = new mongoose.Schema({
  orderId: { type: String, required: true, unique: true },
  userId:  { type: String, required: true, index: true }, // reference
  status:  { type: String, enum: ["PENDING", "PAID", "CANCELLED"], default: "PENDING" },
  total:   { type: Number, required: true },
  shipping: { city: String, pincode: String }, // embedded object
  items: {
    type: [orderItemSchema],
    validate: [(v) => v.length > 0 && v.length <= 100, "1-100 items allowed"], // bounded!
  },
}, { timestamps: true });

// ----- Prisma: normalized relations (Postgres / MySQL) -----
model Order {
  orderId String @id @map("order_id")
  userId String @map("user_id")
  user User @relation(fields: [userId], references: [userId])
  total Decimal @db.Decimal(10, 2)
  status OrderStatus @default(PENDING)
  items OrderItem[]
  payment Payment? // 1:1
  createdAt DateTime @default(now()) @map("created_at")
  @@index([userId, createdAt])
  @@map("orders")
}

model OrderItem {
  orderId String @map("order_id")
  productId String @map("product_id")
  quantity Int
  price Decimal @db.Decimal(10, 2)
  order Order @relation(fields: [orderId], references: [orderId])
  product Product @relation(fields: [productId], references: [productId])
  @@id([orderId, productId]) // composite PK
  @@map("order_items")
}
```

COMMON MISTAKES

- Mongo mein SQL jaisa design (har cheez alag collection) aur phir har API mein 4 \$lookup — Mongo ka fayda khatam.
- Ulta: sab kuch ek document mein embed — unbounded arrays, 16 MB, har chhote update pe bada write.
- Embedded copy (user name in 1 lakh comments) aur user naam change allow — ab 1 lakh docs update karne padenge. Copy sirf un fields ki jo rarely badlein ya snapshot hon.
- SQL mein 'performance ke liye' shuru se denormalize — bina measure kiye. Pehle normalize + index.
- Access patterns likhe bina schema banana. Pehle list banao: 'order page', 'user order history', 'admin daily sales' — phir design.

INTERVIEW QUESTION

Q: E-commerce order ke liye MongoDB schema design karo. Items embed karoge ya reference?

Answer: Items embed karunga with snapshot (productId, name, price, quantity): order hamesha items ke saath padha jaata hai, items bounded hain, order ke saath hi create hote hain, aur single-document write atomic hai. User aur payment reference: user independently exist karta hai aur uske orders unbounded hain (child side pe userId + index); payment ka alag lifecycle hai (retries, webhooks). Product mein catalog display ke liye categoryName extended reference aur rating counters computed pattern se.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• Access-pattern-first thinking • Embed vs reference rules • Unbounded array anti-pattern • Snapshot / extended reference • Order + items: SQL design aur Mongo design dono	• Subset pattern • Computed pattern (counters) • Mongoose subdocument <code>_id</code>: false	• Bucket, polymorphic, schema versioning patterns ki detail • EAV modeling

PART 10 — Indexes

10.1 Index kya hai aur reads fast kyun hote hain?

► CONCEPT

Index = kitaab ke peeche wala **index page**. 'Transactions' topic dhoondhna hai — 500 pages palatne ki jagah index mein dekho: 'Transactions → page 312'. Seedha wahan jao.

Database index = kisi column ki values ka **alag, sorted structure**, jismein har value ke saath row/document ka pointer hota hai.

Output

```
orders table (insert order mein, disk pe bikhra hua)      Index on user_id (sorted)
row#1  01001 1103 ...                                     1103 → row#1, row#3
row#2  01002 1104 ...                                     1104 → row#2
row#3  01003 1103 ...                                     1105 → row#4
row#4  01004 1105 ...

WHERE user_id = '1104'
  Bina index: har row check karo      → 50 lakh rows = 50 lakh checks      (Full scan)
  Index se:   sorted list mein jump    → ~22 steps (log2 of 50 lakh) + 1 fetch
```

► WHY FAST?

Sorted data mein search = **binary search jaisa** (dictionary mein word dhoondhna). Actual structure **B-tree** hai — bas itna jaanna kaafi hai ki ye sorted hota hai aur lookup $O(\log n)$ hai. Sorted hone ki wajah se index **range** (> 500) aur **sort** (**ORDER BY**) mein bhi madad karta hai.

► SQL

SQL (dono)

```
CREATE INDEX idx_orders_user_id ON orders (user_id);
DROP INDEX idx_orders_user_id;                -- PostgreSQL
DROP INDEX idx_orders_user_id ON orders;       -- MySQL (table naam dena padta hai)

-- Indexes dekhna
-- PostgreSQL (psql): \d orders
-- MySQL:             SHOW INDEX FROM orders;
```

► MONGODB

MongoDB

```
db.orders.createIndex({ userId: 1 })           // 1 = ascending, -1 = descending
// 'userId_1' ← auto-generated index name

db.orders.getIndexes()
// [ { v: 2, key: { _id: 1 }, name: '_id_' },    ← _id pe hamesha index hota hai
//   { v: 2, key: { userId: 1 }, name: 'userId_1' } ]

db.orders.dropIndex("userId_1")
```

► COST — INDEX FREE NAHI HAI

Cost	Kyun
Writes slow	Har INSERT/UPDATE/DELETE pe har related index bhi update hota hai. 8 indexes = 1 insert pe 9 writes.

Cost	Kyun
Storage + RAM	Index disk pe jagah leta hai; fast rehne ke liye RAM (cache) mein fit hona chahiye. Bade indexes cache se data ko dhakel dete hain.
Planner confusion	Bahut saare similar indexes → DB kabhi galat index chun leta hai.
Maintenance	Unused indexes bhi write cost dete rehte hain.

Rule: index **queries ke liye** banao, columns ke liye nahi. 'Har column pe index' = galat. Pehle likho ki app kaunsi queries chalati hai (Part 9 wale access patterns), phir unke liye index.

COMMON MISTAKES

- Har column pe index 'safe side' ke liye → writes slow, RAM waste.
- Low-cardinality column pe akela index (**status** — sirf 3 values). 50 lakh mein 25 lakh PAID → index se koi fayda nahi, DB full scan hi karega. Compound index mein kaam aata hai.
- Column pe function lagana: **WHERE LOWER(email) = 'x' / WHERE DATE(created_at) = '...' → normal index use nahi hoga (10.6).**

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• Index = sorted lookup structure • Reads fast, writes slow • Queries ke liye index • <code>_id</code> pe default index	• $O(\log n)$ • Index RAM mein fit hona chahiye	• B-tree page splits, fill factor, internal node structure

10.2 Single-field index

SQL (dono)

```
CREATE UNIQUE INDEX idx_users_email ON users (email);      -- (10.4 mein unique)
CREATE INDEX idx_products_category ON products (category_id);
CREATE INDEX idx_orders_created ON orders (created_at DESC);
```

MongoDB

```
db.products.createIndex({ categoryId: 1 })
db.orders.createIndex({ createdAt: -1 })
```

Single-field index mein direction (1 / -1) matter **nahi** karta — DB index ko dono taraf padh sakta hai. Compound index mein matter karta hai (10.3).

Array field pe index (Mongo): **db.products.createIndex({ tags: 1 })** — har array element ki alag entry banti hai (**multikey index**). **{ tags: "steel" }** fast ho jaata hai. Bade arrays = bada index.

10.3 Compound index + ESR rule

► CONCEPT

Ek index mein **multiple fields**, ek specific order mein. Phone directory jaisa: pehle surname se sorted, same surname mein first name se.

► REQUIREMENT

User order history API: **'User 1103 ke orders, jinka total 500 se zyada, latest pehle'**.

SQL (dono)

```
SELECT order_id, total, created_at
FROM orders
WHERE user_id = '1103' AND total > 500
ORDER BY created_at DESC
LIMIT 20;
```

MongoDB

```
db.orders.find({ userId: "1103", total: { $gt: 500 } })
.sort({ createdAt: -1 }).limit(20)
```

► ESR RULE — COMPOUND INDEX MEIN FIELDS KA ORDER

Order	Type	Hamare query mein	Kyun is position pe
1. Equality	=, \$eq, (chhota \$in)	user_id / userId	Exact value → index ka ek chhota continuous hissa select ho jaata hai
2. Sort	ORDER BY / .sort()	created_at / createdAt	Us hisse ke andar data already sorted milta hai → alag sort step nahi
3. Range	>, <, BETWEEN, \$gt, \$lt, \$ne, regex	total	Range pehle aayi to aage ka sort order toot jaata hai

SQL (dono)

```
CREATE INDEX idx_orders_user_created_total
ON orders (user_id, created_at DESC, total);
```

MongoDB

```
db.orders.createIndex({ userId: 1, createdAt: -1, total: 1 })
```

► GALAT ORDER KYUN GALAT HAI — VISUALIZE KARO

Output

```
✓ Index (userId, createdAt, total) ← ESR
  userId=1103:
    2026-09-10 1800 ✓
    2026-09-05 280  ✗ (total filter)
    2026-09-01 1500 ✓
    ... already createdAt DESC mein sorted
  → Index order mein padho, filter karo, 20 mile → STOP. Sort free.

✗ Index (userId, total, createdAt) ← ERS (galat)
  userId=1103:
    total 280  2026-09-05  ✗
    total 1500 2026-09-01  ✓
    total 1800 2026-09-10  ✓
    ... total se sorted hai, createdAt se NAHI
  → Saari matching rows nikaalo, phir MEMORY MEIN SORT
    (Mongo: SORT stage, MySQL: Using filesort, PG: Sort node)
```

► LEFTMOST PREFIX RULE

Index (`user_id`, `created_at`, `total`) in queries mein kaam aata hai:

Query filter / sort	Index use?	Kyun
<code>user_id = ?</code>	✓	Leftmost field
<code>user_id = ? ORDER BY created_at DESC</code>	✓ (best)	Prefix + sort
<code>user_id = ? AND created_at > ?</code>	✓	Prefix
<code>created_at > ?</code> (user_id nahi)	✗ (mostly)	Pehla field missing — jaise surname ke bina directory
<code>total > 500</code> akela	✗	Pehla field missing

Isliye (`a`, `b`) index hai to alag se (`a`) index ki zaroorat nahi (redundant). Lekin (`b`) ki ho sakti hai.

► SORT DIRECTION

Index { `userId`: 1, `createdAt`: -1 } → `sort({ createdAt: -1 })` aur `sort({ createdAt: 1 })` dono (`userId` equality ke saath) chal jaate hain. Multi-field sort mein index ka pattern ya uska **poora ulta** hi use hota hai: {`a`:1, `b`:-1} index {`a`:1, `b`:-1} aur {`a`:-1, `b`:1} sort support karta hai, {`a`:1, `b`:1} nahi.

COMMON MISTAKES

- Compound index mein range field ko sort field se pehle rakhna → in-memory sort.
- Har query ke liye alag compound index — 15 indexes. Queries group karo, ek index kai queries serve kar sakta hai (prefix rule).
- Redundant indexes: (`user_id`) aur (`user_id`, `created_at`) dono — pehla bekaar.

INTERVIEW QUESTION

Q: Query: status = 'PAID', ORDER BY created_at DESC, total > 1000. Compound index kaise banaoge?

Answer: ESR: (status, created_at DESC, total). Equality (status) pehle taaki PAID wala hissa mile, sort (created_at) doosre taaki us hisse mein already sorted data mile aur LIMIT pe jaldi stop ho, range (total) aakhri mein — index se filter ho jaata hai bina sort tode. (status, total, created_at) banate to total range ke baad created_at order toot jaata aur in-memory sort lagta.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- Compound index - ESR rule — explain kar sako - Leftmost prefix - Redundant index pehchanna	- Sort direction rule - Multikey index	- Index intersection - Skip scan (PG/MySQL 8 edge cases)

10.4 Unique index

SQL (dono)

```
CREATE UNIQUE INDEX idx_users_email ON users (email);
-- (Column pe UNIQUE constraint bhi yahi karta hai — andar unique index banata hai)

INSERT INTO users (user_id, name, email) VALUES ('1107', 'X', 'sonu@shop.in');
-- PostgreSQL: ERROR: duplicate key value violates unique constraint "idx_users_email"
-- MySQL:      ERROR 1062 (23000): Duplicate entry 'sonu@shop.in' for key 'idx_users_email'

-- Composite unique: ek user ek product ka ek hi review de
CREATE UNIQUE INDEX idx_reviews_user_product ON reviews (user_id, product_id);
```

MongoDB

```
db.users.createIndex({ email: 1 }, { unique: true })
db.users.createIndex({ userId: 1 }, { unique: true }) // business ID ke liye (PDF 1 mein kaha tha)
db.reviews.createIndex({ userId: 1, productId: 1 }, { unique: true })
// Duplicate → MongoServerError: E11000 duplicate key error ... dup key: { email: "sonu@shop.in" }
```

► NULL / MISSING AUR UNIQUE — TEENO DB ALAG

DB	Do rows mein email NULL / missing	Solution
PostgreSQL	Allowed (NULL ≠ NULL)	PG 15+: UNIQUE NULLS NOT DISTINCT agar rokna ho
MySQL	Allowed	—
MongoDB	Error! Missing field ko null maana, dusra null duplicate	partialFilterExpression ya sparse: true

► PARTIAL UNIQUE INDEX — SOFT DELETE KA SOLUTION (PDF 1 MEIN PROMISE KIYA THA)

PostgreSQL

```
-- Sirf active (deleted_at IS NULL) users mein email unique
CREATE UNIQUE INDEX idx_users_email_active ON users (email) WHERE deleted_at IS NULL;
```

MySQL

```
-- MySQL mein partial index nahi hai. Trick: generated column (NULL values unique check se bahar)
ALTER TABLE users ADD COLUMN active_email VARCHAR(150)
  AS (IF(deleted_at IS NULL, email, NULL)) STORED;
CREATE UNIQUE INDEX idx_users_active_email ON users (active_email);
```

MongoDB

```
// Soft delete ke liye boolean flag rakho: delete pe isDeleted: true set karo
db.users.createIndex(
  { email: 1 },
  { unique: true, partialFilterExpression: { isDeleted: false } }
)
// Uniqueness sirf isDeleted: false wale docs mein. (partialFilterExpression mein
// sirf limited operators allowed hain – $eq, $exists: true, $gt/$lt, $type, $and...)
```

► NODE.JS — DUPLICATE ERROR HANDLE KARNA

Node.js

```
const DUPLICATE_CODES = new Set([
  11000,           // MongoDB
  "23505",         // PostgreSQL (unique_violation)
  "ER_DUP_ENTRY",  // mysql2
  "P2002",         // Prisma
]);

try {
  await repo.createUser(data);
} catch (err) {
  if (DUPLICATE_CODES.has(err.code)) {
    return res.status(409).json({ error: "Email already exists" });
  }
  throw err;
}
```

Why unique index > 'pehle check karo phir insert'

- App code: `if (await findByEmail(e)) throw; await insert()` — do signup requests ek saath aayi, dono ne check kiya (koi nahi mila), dono ne insert kiya → **duplicate**.
- Unique index DB level pe race-proof hai. App check sirf nice error message ke liye — **asli guard index hai**. (Part 13 ka preview.)
- Payments mein `idempotency_key` / `gateway_payment_id` pe unique index = double charge / double entry se bachav.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- Unique index = race-proof duplicate guard - Error codes: 11000 / 23505 / ER_DUP_ENTRY / P2002 → 409 - Composite unique	- NULL behaviour differences - Partial unique (PG) / generated column (MySQL) / partialFilterExpression	sparse vs partial ki poori comparison

10.5 TTL index — data apne aap expire

► CONCEPT

OTP, password reset tokens, sessions, temporary carts — ek time ke baad bekaar. TTL index MongoDB ko bolta hai: **'itne seconds baad ye documents khud delete kar do'**.

MongoDB

```
db.otp.createIndex({ createdAt: 1 }, { expireAfterSeconds: 300 }) // 5 minute baad delete
db.otp.insertOne({ phone: "9800000001", code: "482913", createdAt: new Date() })

// Har document ka alag expiry time chahiye → expireAfterSeconds: 0 + field mein exact time
db.sessions.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 0 })
db.sessions.insertOne({ userId: "1103", token: "...", expiresAt: new Date(Date.now() + 7*86400e3) })
```

- Field **Date** type ka hona chahiye (string/number pe TTL kaam nahi karta, silently).
- Background job ~**har 60 second** chalta hai — exact second pe delete nahi hota. Isliye code mein bhi check karo: `{ expiresAt: { $gt: new Date() } }`.
- Sirf single-field index pe TTL (compound pe nahi).

► SQL MEIN TTL NAHI HAI — ALTERNATIVES

SQL (dono)

```
-- 1) Scheduled cleanup job (Node cron / pg_cron / MySQL EVENT)
DELETE FROM otps WHERE created_at < NOW() - INTERVAL '5 minutes'; -- PostgreSQL
DELETE FROM otps WHERE created_at < NOW() - INTERVAL 5 MINUTE; -- MySQL

-- 2) Query mein hi expiry check (cleanup chahe late ho)
SELECT * FROM otps WHERE phone = $1 AND created_at > NOW() - INTERVAL '5 minutes';

-- 3) Bahut bade time-series data ke liye: date-wise partitions, purana partition DROP (Part 16)
```

Bade table pe ek saath lakhon rows DELETE karna DB ko bhaari padta hai — batches mein delete karo (**LIMIT 1000** loop). Aur sach bolo to OTP/sessions ke liye aksar **Redis** (built-in key expiry) use hota hai.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- TTL index use case - Date field required ~60s delay → code mein bhi check - SQL: cron cleanup	- expireAfterSeconds: 0 pattern - Redis for OTP/sessions	TTL monitor internals

10.6 Aur kaam ke index types (GOOD TO KNOW)

Type	SQL	MongoDB	Kab
Partial	PG: <code>CREATE INDEX ... WHERE status = 'PENDING'</code> (MySQL: nahi)	<code>partialFilterExpression</code>	Sirf chhote subset pe queries (pending orders)
Expression / functional	PG: <code>ON users (LOWER(email))</code> , MySQL 8.0.13+: <code>((LOWER(email)))</code>	Collation index (case-insensitive)	Function lagi query
Covering	PG: <code>INCLUDE (total)</code> ; MySQL: index mein sab columns	Projection sirf index fields	Index-only read (table touch nahi)

Type	SQL	MongoDB	Kab
Text / full-text	PG: GIN tsvector; MySQL: FULLTEXT	<code>{ name: "text" }</code>	Search box (basic)
JSON / array	PG: GIN on JSONB/array	Multikey (auto)	Array/JSON field queries
Geo	PostGIS	<code>2dsphere</code>	'Near me' queries

► COVERED QUERY EXAMPLE

MongoDB

```
db.orders.createIndex({ userId: 1, createdAt: -1, total: 1 })
db.orders.find({ userId: "1103" }, { _id: 0, createdAt: 1, total: 1 }) // sab fields index mein
// explain → IXSCAN, totalDocsExamined: 0 ← documents padhe hi nahi! (Part 11)
```

10.7 Production mein index banana + Node.js

PostgreSQL

```
-- Bade live table pe normal CREATE INDEX writes ko block kar sakta hai
CREATE INDEX CONCURRENTLY idx_orders_user_created ON orders (user_id, created_at DESC);
-- (Transaction ke andar nahi chal sakta; fail ho to INVALID index bachta hai – drop karke retry)
```

MySQL

```
-- InnoDB online DDL: zyadatar index creation writes block nahi karta
ALTER TABLE orders ADD INDEX idx_orders_user_created (user_id, created_at),
ALGORITHM=INPLACE, LOCK=NONE;
```

MongoDB

```
// MongoDB 4.2+: index build sirf shuru aur end mein short lock leta hai. Phir bhi peak time avoid karo.
db.orders.createIndex({ userId: 1, createdAt: -1 })
```

Node.js

```
// ----- Mongoose -----
orderSchema.index({ userId: 1, createdAt: -1, total: 1 }); // compound
userSchema.path("email").index({ unique: true });
otpSchema.index({ createdAt: 1 }, { expireAfterSeconds: 300 }); // TTL

mongoose.connect(url, { autoIndex: process.env.NODE_ENV !== "production" });
// Production mein autoIndex OFF: har app start pe index build trigger na ho.
// Indexes migration script / deploy step mein banao: await Order.syncIndexes()
// (careful: syncIndexes schema mein na likhe extra indexes DROP kar deta hai)

// ----- Prisma (schema.prisma) -----
// @@index([userId, createdAt(sort: Desc), total])
// email String @unique
// → prisma migrate dev (migration SQL file banti hai, review karo)

// ----- Sequelize migration -----
await queryInterface.addIndex("orders", ["user_id", "created_at", "total"],
  { name: "idx_orders_user_created_total", concurrently: true }); // concurrently: PG only
```

Unused indexes dhoondhna (GOOD TO KNOW): PostgreSQL `pg_stat_user_indexes` (`idx_scan = 0`), MySQL `sys.schema_unused_indexes`, MongoDB `db.orders.aggregate([{$indexStats: {}}])`.

COMMON MISTAKES

- Mongoose unique ko validation samajhna — `unique: true` sirf index banata hai; agar collection mein pehle se duplicates hain to index build fail, aur `autoIndex` off hai to index bana hi nahi.
- Production mein bade table pe bina `CONCURRENTLY` index — writes ruk gaye, API timeouts.
- Index banaya lekin query ne use nahi kiya (function, type mismatch, galat field order) — hamesha `EXPLAIN` se verify (Part 11).

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• Index in ORM/ODM schema • Production: <code>CONCURRENTLY</code> / <code>online</code> / <code>autoIndex</code> off • <code>EXPLAIN</code> se verify	• Partial, expression, covering indexes • Unused index detection	• GIN/GiST internals • Index build internals

Part 10 — Memory Table

Kaam	SQL	MongoDB
Single	<code>CREATE INDEX i ON t (c)</code>	<code>createIndex({ c: 1 })</code>
Compound	<code>ON t (a, b DESC, c)</code>	<code>{ a: 1, b: -1, c: 1 }</code>
Unique	<code>CREATE UNIQUE INDEX / UNIQUE</code>	<code>{ unique: true }</code>
Partial	<code>PG: ... WHERE cond</code>	<code>{ partialFilterExpression }</code>
TTL	<code>Cron DELETE / partitions</code>	<code>{ expireAfterSeconds }</code>
Case-insensitive	<code>PG: ON (LOWER(c)); MySQL: default ci</code>	<code>collation index</code>
List	<code>\d t / SHOW INDEX FROM t</code>	<code>getIndexes()</code>
Online build	<code>CONCURRENTLY / ALGORITHM=INPLACE</code>	Default (4.2+)
Compound order	<code>ESR: Equality → Sort → Range</code>	Same
Dup error	<code>23505 / ER_DUP_ENTRY</code>	11000

PART 11 — EXPLAIN / Query Performance

11.1 EXPLAIN kya hai?

► CONCEPT

EXPLAIN database se poochta hai: **'Ye query tum kaise chalaoge?'** — index use karoge ya poori table padhoge, kitni rows padhoge, sort kahan karoge. Google Maps ka route preview jaisa: chalne se pehle dekh lo kaunsa raasta lega.

Command	Kya karta hai	Query chalti hai?
EXPLAIN	Plan + estimates (andaaza)	Nahi
EXPLAIN ANALYZE	Plan + actual time aur rows	Haan!
explain("queryPlanner") (Mongo)	Winning plan	Nahi
explain("executionStats") (Mongo)	Plan + actual docs/keys examined, time	Haan

Warning

- EXPLAIN ANALYZE DELETE ... / UPDATE sach mein data badal dega. Safe tareeka: BEGIN; EXPLAIN ANALYZE UPDATE ...; ROLLBACK;
- Mongo executionStats bhi query chalata hai — production pe heavy query ka explain bhi load daalta hai.

► SCENARIO JO POORE PART MEIN USE KARENGE

Shop badi ho gayi: **orders mein 50 lakh rows**. API GET /users/1103/orders (latest 20) ab 2 second le rahi hai. Kyun? Dekhte hain.

SQL (dono)

```
SELECT order_id, total, status, created_at
FROM orders WHERE user_id = '1103'
ORDER BY created_at DESC LIMIT 20;
```

MongoDB

```
db.orders.find({ userId: "1103" }, { orderId: 1, total: 1, status: 1, createdAt: 1 })
.sort({ createdAt: -1 }).limit(20)
```

11.2 MongoDB — COLLSCAN vs IXSCAN

► BINA INDEX

MongoDB

```
db.orders.find({ userId: "1103" }).sort({ createdAt: -1 }).limit(20).explain("executionStats")
```

Output

```
{
  queryPlanner: {
    winningPlan: {
      stage: "SORT",
      sortPattern: { createdAt: -1 }, limitAmount: 20,
      inputStage: {
        stage: "COLLSCAN",
        filter: { userId: { $eq: "1103" } },
        direction: "forward"
      }
    }
  },
  executionStats: {
    nReturned: 20,
    executionTimeMillis: 2140,
    totalKeysExamined: 0,
    totalDocsExamined: 5000000
  }
}
```

← memory mein sort
← POORI collection padhi
← koi index use nahi
← 50 lakh docs padhe, 20 lautaye

► INDEX BNAO AUR DOBARA DEKHO

MongoDB

```
db.orders.createIndex({ userId: 1, createdAt: -1 })
```

Output

```
{
  queryPlanner: {
    winningPlan: {
      stage: "LIMIT", limitAmount: 20,
      inputStage: {
        stage: "FETCH",
        inputStage: {
          stage: "IXSCAN",
          keyPattern: { userId: 1, createdAt: -1 },
          indexName: "userId_1_createdAt_-1",
          direction: "forward",
          indexBounds: { userId: [ '"1103"', "1103" ], createdAt: [ "[MaxKey, MinKey]" ] }
        }
      }
    }
  },
  executionStats: {
    nReturned: 20,
    executionTimeMillis: 1,
    totalKeysExamined: 20,
    totalDocsExamined: 20
  }
}
```

← index se mile docs laao
← index scan
← 20 padhe, 20 lautaye. PERFECT.

► STAGES KA MATLAB

Stage	Matlab	Achha / Bura
COLLSCAN	Poori collection scan	Bade collection pe bura (chhote pe theek)
IXSCAN	Index scan	Achha (lekin keysExamined dekho)
FETCH	Index se mile pointers se document padhna	Normal
SORT	Memory mein sort (index sort nahi de paya)	Bade data pe bura — ESR check karo

Stage	Matlab	Achha / Bura
PROJECTION_COVERED	Sab fields index se, doc padha hi nahi	Best
SORT_KEY_GENERATOR	In-memory sort ki taiyari	SORT ka saathi — dhyan do

► TEEN NUMBERS JO SABSE ZYADA BATATE HAIN

Output

```

nReturned          → kitne docs lautaye
totalKeysExamined  → kitni index entries padhi
totalDocsExamined  → kitne documents padhe

IDEAL:  nReturned ≈ totalKeysExamined ≈ totalDocsExamined
BURA:   totalDocsExamined >> nReturned  (5000000 vs 20) → missing / bad index
BURA:   totalKeysExamined >> nReturned  (48000 vs 20)  → index hai, par galat order (ESR)
BEST:    totalDocsExamined = 0              → covered query

```

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
- COLLSCAN vs IXSCAN - nReturned / keysExamined / docsExamined ratio - SORT stage = in-memory sort	- FETCH, PROJECTION_COVERED - indexBounds padhna	- rejectedPlans ka poora analysis - Plan cache internals

11.3 PostgreSQL — EXPLAIN ANALYZE

► BINA INDEX

PostgreSQL

```

EXPLAIN ANALYZE
SELECT order_id, total, status, created_at
FROM orders WHERE user_id = '1103'
ORDER BY created_at DESC LIMIT 20;

```

Output

```

Limit  (cost=108452.1..108452.2 rows=20 width=40) (actual time=812.4..812.5 rows=20 loops=1)
  -> Sort  (cost=108452.1..108452.3 rows=48 width=40) (actual time=812.4..812.4 rows=20 loops=1)
        Sort Key: created_at DESC
        Sort Method: top-N heapsort  Memory: 27kB
  -> Seq Scan on orders  (cost=0.00..108450.0 rows=48 width=40)
        (actual time=0.03..811.9 rows=52 loops=1)
        Filter: ((user_id)::text = '1103'::text)
        Rows Removed by Filter: 4999948      ← 50 lakh padhe, 52 kaam ke
Planning Time: 0.11 ms
Execution Time: 812.6 ms

```

► INDEX KE BAAD

PostgreSQL

```

CREATE INDEX CONCURRENTLY idx_orders_user_created ON orders (user_id, created_at DESC);

```

Output

```
Limit (cost=0.43..12.8 rows=20 width=40) (actual time=0.04..0.09 rows=20 loops=1)
-> Index Scan using idx_orders_user_created on orders
    (cost=0.43..30.1 rows=48 width=40) (actual time=0.04..0.08 rows=20 loops=1)
    Index Cond: ((user_id)::text = '1103'::text)
Planning Time: 0.15 ms
Execution Time: 0.12 ms                                     ← 812 ms → 0.12 ms
```

► KAISE PADHEIN

- Plan **andar se bahar** padho — sabse indented line pehle chalti hai (Seq Scan → Sort → Limit).
- **cost=0.43..12.8** — planner ka andaaza (units, ms nahi). **actual time** — asli ms. **rows=48** estimate vs **rows=52** actual.
- **Rows Removed by Filter** — bada number = bekaar kaam = index ki zaroorat.
- **loops** — ye node kitni baar chala (JOIN mein important: actual time × loops).
- Estimate aur actual mein **bahut** farak (rows=10 vs rows=500000) → table statistics purane → **ANALYZE orders;**.

Node	Matlab
Seq Scan	Poori table (COLLSCAN jaisa)
Parallel Seq Scan + Gather	Poori table, kai workers — phir bhi full scan
Index Scan	Index se dhoondha, phir table se row laaya (IXSCAN + FETCH)
Index Only Scan	Sirf index se jawab (covered)
Bitmap Index Scan → Bitmap Heap Scan	Kaafi rows match — pehle index se list banayi, phir table se batch mein padha. Normal hai.
Sort (external merge Disk)	Memory kam padi, disk pe sort — slow
Nested Loop / Hash Join / Merge Join	JOIN ke tareeke — naam pehchan lo, detail zaroori nahi

11.4 MySQL — EXPLAIN

MySQL

```
EXPLAIN SELECT order_id, total, status, created_at
FROM orders WHERE user_id = '1103' ORDER BY created_at DESC LIMIT 20;
```

Output

```
Bina index:
Bina index (important columns):
+-----+-----+-----+-----+-----+-----+-----+
| table | type | possible_keys | key | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+
| orders | ALL | NULL          | NULL | 4987231 | 10.00 | Using where; Using filesort |
+-----+-----+-----+-----+-----+-----+-----+

Index (user_id, created_at) ke baad:
+-----+-----+-----+-----+-----+-----+-----+
| table | type | key | ref | rows | Extra |
+-----+-----+-----+-----+-----+-----+-----+
| orders | ref | idx_orders_user_created | const | 52 | Backward index scan |
+-----+-----+-----+-----+-----+-----+-----+
(Asli output mein id, select_type, key_len wagairah columns bhi hote hain)
```

Column	Kya dekho
type	ALL = full scan (bura). index = poora index scan. range = index range. ref = index lookup. const/eq_ref = PK/unique lookup (best).
key	Kaunsa index use hua. NULL = koi nahi.
rows	Kitni rows padhne ka andaaza.
Extra	Using filesort = in-memory/disk sort (SORT stage jaisa). Using temporary = temp table (GROUP BY/DISTINCT). Using index = covered (best).

MySQL 8.0.18+ mein **EXPLAIN ANALYZE** bhi hai (actual times ke saath, tree format). **EXPLAIN FORMAT=TREE** Postgres jaisa padhne mein aasaan.

Backward index scan — index ASC bana tha, query DESC maang rahi thi, MySQL ne index ulta padh liya. Theek hai.

11.5 Problems pehchanna — 5 situations

► 1) MISSING INDEX

Sign	SQL	Mongo
Full scan	Seq Scan / type: ALL	COLLSCAN
Bahut rows phenki	Rows Removed by Filter: 4999948	totalDocsExamined: 5000000, nReturned: 20

Fix: query ke WHERE + ORDER BY ke hisaab se (ESR) index.

► 2) BAD INDEX (INDEX HAI, PAR GALAT)

MongoDB

```
// Index: { userId: 1, total: 1, createdAt: -1 }      ← ERS (galat)
// Query: { userId: "2001", total: { $gt: 500 } } sort { createdAt: -1 } limit 20
// (2001 = bada B2B customer, 50,000 orders)

executionStats: {
  nReturned: 20,
  totalKeysExamined: 48000,          ← 48k index entries padhi
  totalDocsExamined: 48000,
  executionTimeMillis: 380
}
winningPlan: SORT ← FETCH ← IXSCAN          ← SORT stage = in-memory sort

// Fix: { userId: 1, createdAt: -1, total: 1 }      (ESR)
// → keysExamined ~25, docsExamined 20, no SORT stage, 2 ms
```

SQL mein same sign: **Index Scan** ke upar **Sort** node / MySQL **Using filesort**, aur index scan ke rows >> result rows.

Doosre 'bad index' cases: low-selectivity index choose hua (**status** index se 25 lakh PAID rows), ya function/type mismatch ki wajah se index ignore (**WHERE LOWER(email)=...**, string column vs number parameter MySQL mein).

► 3) TOO MANY DOCUMENTS SCANNED (INDEX HAI, LEKIN QUERY HI BROAD HAI)

MongoDB

```
db.orders.find({ status: { $ne: "CANCELLED" } }).sort({ createdAt: -1 }) // no limit!
// $ne / $nin index ka bahut bada hissa scan karate hain + koi limit nahi → 48 lakh docs
```

Fix: query redesign — positive condition (`$in: ["PAID", "PENDING"]`), pagination/limit, date range, ya pre-aggregated data.

► 4) LARGE DOCUMENTS / WIDE ROWS

Plan achha hai (IXSCAN, docsExamined = nReturned), phir bhi slow. Kyun? Har document **2 MB** ka hai (unbounded array, base64 images, logs embedded). 20 docs = 40 MB network pe.

MongoDB

```
db.orders.stats().avgObjSize // average document size (bytes)
Object.bsonsize(db.orders.findOne({ orderId: "01001" })) // ek document ka size
db.orders.aggregate([{ $project: { s: { $bsonSize: "$$ROOT" } } }, { $sort: { s: -1 } }, { $limit: 5 }])
```

PostgreSQL

```
SELECT pg_size_pretty(pg_total_relation_size('orders')); -- table + indexes size
SELECT order_id, pg_column_size(t.*) FROM orders t ORDER BY 2 DESC LIMIT 5; -- badi rows
```

Fix: projection (sirf zaroori fields), bade arrays alag collection mein (Part 9), files S3 mein aur DB mein sirf URL, **SELECT *** band.

► 5) SLOW QUERY — PEHLE PATA KAISE CHALE?

DB	Slow query log	Top queries
PostgreSQL	<code>log_min_duration_statement = 500</code> (ms)	<code>pg_stat_statements</code> extension
MySQL	<code>slow_query_log = ON</code> , <code>long_query_time = 0.5</code>	<code>sys.statement_analysis</code> / <code>performance_schema</code>
MongoDB	mongod log mein default >100 ms queries; <code>db.setProfilingLevel(1, { slowms: 100 })</code>	<code>db.system.profile</code> , Atlas Performance Advisor

PostgreSQL

```
-- Sabse zyada total time khane wali 5 queries
SELECT calls, round(mean_exec_time::numeric, 1) AS avg_ms, round(total_exec_time::numeric) AS total_ms,
       left(query, 80) AS query
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 5;
```

MongoDB

```
db.system.profile.find({ millis: { $gt: 100 } }).sort({ ts: -1 }).limit(5)
// Har entry: op, ns (collection), command, keysExamined, docsExamined, millis, planSummary: "COLLSCAN"
```

Tip: 'sabse slow' query se zyada important aksar 'sabse zyada total time' wali hoti hai — 20 ms ki query jo 1 lakh baar/min chalti hai.

11.6 Node.js — performance visibility

Node.js

```
// ----- Har query ka time log karo (pg wrapper) -----
async function timedQuery(sql, params) {
  const start = process.hrtime.bigint();
  try { return await pool.query(sql, params); }
  finally {
    const ms = Number(process.hrtime.bigint() - start) / 1e6;
    // params mat log karo (PII)
    if (ms > 200) logger.warn({ ms, sql: sql.slice(0, 200) }, "slow query");
  }
}

// ----- Timeouts — ek buri query poora DB na bithaa de -----
const pool = new Pool({ connectionString: PG_URL, statement_timeout: 5000 }); // PG: 5s
await mysqlPool.query("SELECT /*+ MAX_EXECUTION_TIME(5000) */ ..."); // MySQL: SELECT only
await orders.find(filter).maxTimeMS(5000).toArray(); // Mongo

// ----- Dev mein queries dekho -----
mongoose.set("debug", true); // har query console pe
const prisma = new PrismaClient({ log: [{ emit: "event", level: "query" }] });
prisma.$on("query", (e) => e.duration > 200 && console.warn(e.duration, e.query));

// ----- Code se explain (test / debugging) -----
const plan = await orders.find({ userId: "1103" }).sort({ createdAt: -1 }).limit(20)
  .explain("executionStats");
console.log(plan.queryPlanner.winningPlan, plan.executionStats.totalDocsExamined);
```

► DEBUG CHECKLIST — API SLOW HAI

Output

1. Kaunsa endpoint slow? (APM / logs: p95 latency)
2. Us request mein kitni queries? Kaunsi slow? (N+1 to nahi? — Part 19)
3. Slow query ka EXPLAIN ANALYZE / explain("executionStats")
4. Full scan? → index (ESR)
Index hai, SORT stage? → index order galat
keys/docs >> returned? → selective index / query redesign
Plan theek, fir bhi slow? → document size / network / locks / DB CPU
5. Fix karo → EXPLAIN dobara → production metrics mein verify

COMMON MISTAKES

- Chhote dev database (100 rows) pe EXPLAIN dekh ke 'sab theek hai' — 100 rows pe Seq Scan hi best hai. Production jitna data leke test karo.
- Seq Scan dekhte hi panic — agar query table ka bada hissa maang rahi hai (30%+), to full scan index se fast hota hai. Planner sahi hai.
- Index banaya, verify nahi kiya — EXPLAIN mein index dikhna chahiye.
- Production pe EXPLAIN ANALYZE UPDATE/DELETE bina ROLLBACK.
- Query logs mein parameters (email, phone) log karna — privacy issue.

INTERVIEW QUESTION

Q: Production mein ek API slow hai. Database side pe kaise debug karoge?

Answer: Pehle identify: APM/slow query log/pg_stat_statements/Mongo profiler se kaunsi query slow ya sabse zyada total time le rahi hai, aur request mein queries ki count (N+1). Phir EXPLAIN ANALYZE / explain('executionStats'): full scan (Seq Scan/COLLSCAN) → ESR index; SORT stage/filesort → index order; docsExamined >> nReturned → query ya index selective nahi; plan theek to document size, locks, connection pool wait check. Fix ke baad EXPLAIN aur p95 latency se verify.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• EXPLAIN vs EXPLAIN ANALYZE • COLLSCAN/IXSCAN/SORT; Seq Scan/Index Scan; type ALL/ref + filesort • docsExamined vs nReturned ratio • Slow query log exist karta hai • Query timeouts	• pg_stat_statements / profiler • Bitmap scan, Index Only Scan • ANALYZE for stale stats • Document size check	• Cost formula • Join algorithm internals • Planner GUC tuning • Plan cache internals

Part 11 — Memory Table

Matlab	PostgreSQL	MySQL	MongoDB
Plan dekho	EXPLAIN	EXPLAIN	explain('queryPlanner')
Actual run	EXPLAIN ANALYZE	EXPLAIN ANALYZE (8.0.18+)	explain('executionStats')
Full scan	Seq Scan	type: ALL	COLLSCAN
Index use	Index Scan	type: ref/range/const	IXSCAN + FETCH
Covered	Index Only Scan	Extra: Using index	docsExamined: 0
Memory sort	Sort node	Using filesort	SORT stage
Bekaar kaam	Rows Removed by Filter	rows >> result	docsExamined >> nReturned
Slow log	log_min_duration_stateme nt	slow_query_log	profiler / slowms
Timeout	statement_timeout	MAX_EXECUTION_TIME	maxTimeMS

PART 12 — Transactions

12.1 Problem: Place Order

► BUSINESS SCENARIO

Sonu (1103) 3 Steel Pipe (P101) khareed raha hai, UPI se. Backend ko 4 kaam karne hain:

Output			
1. orders	mein naya order	01005	(total 1500, status PAID)
2. order_items	mein line	01005, P101,	qty 3, price 500
3. products	ka stock kam	P101:	20 → 17
4. payments	mein entry	PAY503, 01005,	1500, UPI, SUCCESS

► WHY? — BEECH MEIN FAIL HUA TO KYA HOGA

Kahan fail	Bina transaction result	Business nuksaan
Step 2 ke baad server crash	Order + items hain, stock kam nahi hua	Stock galat — oversell hoga
Step 3 ke baad DB error	Stock kam, payment record nahi	Paisa aaya ya nahi — pata nahi; reconciliation nightmare
Step 3 pe stock kam pada	Order ban chuka hai, stock nahi	Customer ko order confirmation, maal nahi

Chahiye: **ya to chaaron ho, ya ek bhi nahi**. Yahi **transaction** hai — multiple operations ko ek unit bana dena.

12.2 ACID — simple Hinglish mein

Letter	Matlab	Place Order mein
A tomicity	Sab ya kuch nahi. Beech mein fail → sab undo (rollback).	Payment insert fail → order, items, stock change sab wapas.
C onsistency	Transaction ke baad data rules follow karta hai (FK, CHECK, unique, business rules).	Stock negative nahi ho sakta (CHECK stock >= 0); order_items ka order exist karega.
I solation	Saath chal rahe transactions ek doosre ka adhoora kaam nahi dekhte.	Priya ki report ko O1005 tab dikhega jab poora commit ho — aadha nahi.
D urability	COMMIT ho gaya matlab data safe — turant crash ho tab bhi.	'Order placed' dikha diya → server restart ke baad bhi order hai.

Durability kaise? DB commit se pehle change ek **log** (Postgres: WAL, MySQL: redo log, Mongo: journal) mein disk pe likhta hai. Bas itna jaanna kaafi hai — internals zaroori nahi.

12.3 SQL Transaction

SQL (dono)

```
BEGIN;                                -- MySQL: START TRANSACTION; (BEGIN bhi chalta hai)

INSERT INTO orders (order_id, user_id, total, status)
VALUES ('01005', '1103', 1500, 'PAID');

INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES ('01005', 'P101', 3, 500);

UPDATE products
SET stock = stock - 3
WHERE product_id = 'P101' AND stock >= 3;      -- stock kam ho to 0 rows update hongi

INSERT INTO payments (payment_id, order_id, amount, method, status)
VALUES ('PAY503', '01005', 1500, 'UPI', 'SUCCESS');

COMMIT;                                -- sab permanent
-- Kuch galat hua to: ROLLBACK;          -- sab undo, jaise kuch hua hi nahi
```

Output

```
BEGIN
INSERT 0 1
INSERT 0 1
UPDATE 1      ← 1 row updated = stock tha. 'UPDATE 0' aata to app ROLLBACK karti.
INSERT 0 1
COMMIT
```

► LINE-BY-LINE

- **BEGIN** — transaction shuru. Ab ke changes sirf is connection ko dikhenge jab tak COMMIT na ho.
- **WHERE ... AND stock >= 3** — **stock check aur update ek hi atomic statement mein.** Pehle SELECT karke check karna race condition hai (Part 13).
- **UPDATE 0 rows** DB ke liye error nahi hai! App ko **rowCount** check karke khud ROLLBACK karna padega.
- **COMMIT** — sab changes ek saath permanent aur doosron ko visible.
- **ROLLBACK** — sab changes discard. Connection toot jaaye (crash) to DB khud rollback karta hai.

► POSTGRESQL VS MYSQL — TRANSACTION KE ANDAR ERROR

PostgreSQL

```
BEGIN;
INSERT INTO orders ... ('01005', ...);
INSERT INTO orders ... ('01005', ...);      -- duplicate key ERROR
INSERT INTO payments ...;
-- ERROR: current transaction is aborted, commands ignored until end of transaction block
ROLLBACK;                                   -- ab sirf ROLLBACK hi chalega
```

MySQL

```
START TRANSACTION;
INSERT INTO orders ... ('01005', ...);
INSERT INTO orders ... ('01005', ...);      -- ERROR 1062 Duplicate entry
INSERT INTO payments ...;                  -- ye CHAL JAAYEGA! (sirf failed statement undo hua)
COMMIT;                                   -- adhoora data commit ho sakta hai agar app ne ROLLBACK nahi kiya
```

Lesson: **error aaye to app code hamesha explicitly ROLLBACK kare** (try/catch). MySQL mein DDL (**CREATE/ALTER TABLE**) transaction ko **implicitly commit** kar deta hai — transaction ke andar DDL mat chalo. Postgres mein DDL bhi transactional hai.

12.4 MongoDB Transaction

► PEHLE SOCHO — KYA TRANSACTION CHAHIYE BHI?

MongoDB mein **ek document ka write hamesha atomic** hai. Hamare design mein order + items + shipping ek document hai → order create karna already atomic. Lekin **stock** (products) aur **payment** (payments) alag collections hain → teen documents, teen collections → **multi-document transaction** chahiye.

Output

SQL mein 4 tables → transaction (orders, order_items, products, payments)	Mongo mein 3 collections → transaction (orders [items embedded], products, payments)
--	---

► MONGOSH

MongoDB

```
const session = db.getMongo().startSession();
const s = session.getDatabase("pipe_shop");

session.startTransaction({ readConcern: { level: "snapshot" }, writeConcern: { w: "majority" } });
try {
  const stock = s.products.updateOne(
    { productId: "P101", stock: { $gte: 3 } }, // check + update atomic
    { $inc: { stock: -3 } }
  );
  if (stock.modifiedCount !== 1) throw new Error("OUT_OF_STOCK");

  s.orders.insertOne({
    orderId: "01005", userId: "1103", total: 1500, status: "PAID",
    items: [ { productId: "P101", name: "Steel Pipe", quantity: 3, price: 500 } ],
    createdAt: new Date()
  });

  s.payments.insertOne({ paymentId: "PAY503", orderId: "01005", amount: 1500,
    method: "UPI", status: "SUCCESS" });

  session.commitTransaction();
} catch (e) {
  session.abortTransaction(); // = ROLLBACK
  throw e;
} finally {
  session.endSession();
}
```

► LINE-BY-LINE

- **startSession()** — transaction ek **session** se bandhi hoti hai (SQL mein connection se).
- **startTransaction()** = **BEGIN**. **commitTransaction()** = **COMMIT**. **abortTransaction()** = **ROLLBACK**.
- **readConcern: snapshot** — transaction ek consistent snapshot dekhta hai. **writeConcern: majority** — commit majority replicas pe likha jaaye (durability, Part 15).
- Stock update **pehle** kiya — sabse zyada fail hone wala step pehle, taaki bekaar kaam na ho.
- **Replica set zaroori hai** — standalone mongod pe transactions nahi chalte. Local dev mein single-node replica set chalo (**mongod --replSet rs0 + rs.initiate()**), ya Docker image

with replset).

► SQL VS MONGODB TRANSACTIONS

	SQL (PG / MySQL)	MongoDB
Kab se	Hamesha se, core feature	4.0 (replica set), 4.2 (sharded)
Design philosophy	Normal hai, har jagah use	Available, lekin pehle embedding se avoid karne ki koshish
Requirement	Kuch nahi	Replica set / sharded cluster
Time limit	Nahi (lekin lambi transactions buri)	Default 60 sec lifetime
Conflict	Locks — doosra wait karta hai (Part 13)	WriteConflict error → poora transaction retry
Performance	Normal	Single-doc writes se mehenga
Error ke baad	PG: aborted state; MySQL: continue (app ROLLBACK kare)	Abort karke retry

12.5 Isolation levels (GOOD TO KNOW, interview mein aata hai)

Problem	Matlab (simple)
Dirty read	Doosre ka uncommitted data padh liya (jo baad mein rollback hua)
Non-repeatable read	Same row do baar padhi, beech mein kisi ne commit karke badal di
Phantom read	Same WHERE do baar chalaya, beech mein nayi rows aa gayi

Level	Dirty	Non-repeatable	Phantom	Default kahan
Read Uncommitted	Possible	Possible	Possible	—
Read Committed	X	Possible	Possible	PostgreSQL
Repeatable Read	X	X	Possible (standard); MySQL/PG mostly rok dete	MySQL InnoDB
Serializable	X	X	X	—

MongoDB transactions snapshot isolation dete hain (readConcern snapshot). Backend engineer ke liye itna kaafi: **defaults jaano, samjho ki higher isolation = zyada safety + zyada conflicts/retries**, aur inventory jaise cases atomic conditional update se solve karo.

SQL (dono)

```
BEGIN ISOLATION LEVEL SERIALIZABLE; -- PostgreSQL: conflict pe "could not serialize" error → retry  
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; -- MySQL: agle transaction ke liye (START se pehle)
```

12.6 Node.js — real service code

► A) PG — SABSE COMMON BUG PEHLE

Node.js

```
// GALAT — pool.query har baar ALAG connection le sakta hai!
await pool.query("BEGIN");
await pool.query("INSERT INTO orders ...");           // shayad connection #2 pe — transaction ke bahar!
await pool.query("COMMIT");                           // shayad connection #3 pe
```

Node.js

```
// SAHI — ek client pool se lo, saara kaam usi pe
async function placeOrderPg({ orderId, userId, productId, qty, paymentId, method }) {
  const client = await pool.connect();
  try {
    await client.query("BEGIN");

    const { rows: [product] } = await client.query(
      `UPDATE products SET stock = stock - $1
       WHERE product_id = $2 AND stock >= $1
       RETURNING price, name`, [qty, productId]);
    if (!product) throw new AppError(409, "OUT_OF_STOCK");

    const total = Number(product.price) * qty;
    await client.query(
      "INSERT INTO orders (order_id, user_id, total, status) VALUES ($1, $2, $3, 'PAID')",
      [orderId, userId, total]);
    await client.query(
      "INSERT INTO order_items (order_id, product_id, quantity, price) VALUES ($1, $2, $3, $4)",
      [orderId, productId, qty, product.price]);
    await client.query(
      `INSERT INTO payments (payment_id, order_id, amount, method, status)
       VALUES ($1, $2, $3, $4, 'SUCCESS')`,
      [paymentId, orderId, total, method]);

    await client.query("COMMIT");
    return { orderId, total };
  } catch (err) {
    await client.query("ROLLBACK");
    throw err;
  } finally {
    client.release();           // BHOOLE to connection leak → pool khaali → app hang
  }
}
```

► B) MYSQL2

Node.js

```
async function placeOrderMysql(input) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const [upd] = await conn.execute(
      "UPDATE products SET stock = stock - ? WHERE product_id = ? AND stock >= ?",
      [input.qty, input.productId, input.qty]);
    if (upd.affectedRows !== 1) throw new AppError(409, "OUT_OF_STOCK");
    const [[p]] = await conn.execute(
      "SELECT price FROM products WHERE product_id = ?", [input.productId]);
    // ... orders, order_items, payments INSERTs (same as above, ? placeholders)
    await conn.commit();
  } catch (err) {
    await conn.rollback();
    throw err;
  } finally {
    conn.release();
  }
}
```

► C) MONGODB DRIVER — WITHTRANSACTION (RETRY KE SAATH)

Node.js

```
async function placeOrderMongo(client, { orderId, userId, productId, qty, paymentId, method }) {
  const db = client.db("pipe_shop");
  const session = client.startSession();
  try {
    let result;
    await session.withTransaction(async () => {
      const product = await db.collection("products").findOneAndUpdate(
        { productId, stock: { $gte: qty } },
        { $inc: { stock: -qty } },
        { session, returnDocument: "after", projection: { name: 1, price: 1 } } // ← session!
      );
      if (!product) throw new AppError(409, "OUT_OF_STOCK"); // throw → auto abort

      const total = product.price * qty;
      await db.collection("orders").insertOne({
        orderId, userId, total, status: "PAID", createdAt: new Date(),
        items: [{ productId, name: product.name, quantity: qty, price: product.price }],
        }, { session }); // ← session!
      await db.collection("payments").insertOne(
        { paymentId, orderId, amount: total, method, status: "SUCCESS" }, { session });
      result = { orderId, total };
    }, { readConcern: { level: "snapshot" }, writeConcern: { w: "majority" } });
    return result;
  } finally {
    await session.endSession();
  }
}

// withTransaction: TransientTransactionError (e.g. WriteConflict) aur UnknownTransactionCommitResult
// pe callback/commit khud RETRY karta hai. Isliye callback idempotent rakho (bahar side-effects nahi –
// email, HTTP call, counter++ callback ke andar mat karo).
```

► D) PRISMA, SEQUELIZE, MONGOOSE

Node.js

```
// ----- Prisma interactive transaction -----
await prisma.$transaction(async (tx) => {
  const { count } = await tx.product.updateMany({
    where: { productId, stock: { gte: qty } }, data: { stock: { decrement: qty } } });
  if (count !== 1) throw new AppError(409, "OUT_OF_STOCK"); // throw → rollback
  await tx.order.create({ data: { orderId, userId, total, status: "PAID",
    items: { create: [{ productId, quantity: qty, price } ] } } });
  await tx.payment.create({ data: { paymentId, orderId, amount: total, method, status: "SUCCESS" } });
}, { timeout: 5000 }); // tx.* use karo, prisma.* nahi (warna transaction ke bahar!)

// ----- Sequelize managed transaction -----
await sequelize.transaction(async (t) => {
  const [n] = await Product.decrement("stock", { by: qty,
    where: { productId, stock: { [Op.gte]: qty } }, transaction: t });
  // ... Order.create(..., { transaction: t }) – har call mein { transaction: t }
});

// ----- Mongoose -----
const session = await mongoose.startSession();
await session.withTransaction(async () => {
  const p = await Product.findOneAndUpdate({ productId, stock: { $gte: qty } },
    { $inc: { stock: -qty } }, { new: true, session });
  if (!p) throw new AppError(409, "OUT_OF_STOCK");
  // create() + session → ARRAY form zaroori
  await Order.create([{ orderId, userId, total, items: [...] }], { session });
  await Payment.create([{ paymentId, orderId, amount: total, method }], { session });
});
await session.endSession();
```

12.7 Production rule: external calls transaction ke BAHAR

Upar ke example mein payment 'SUCCESS' seedha likh diya. Real life mein **payment gateway (Razorpay/Stripe) ek HTTP call hai** jo 2–30 second le sakti hai. Usko transaction ke andar rakha to itni der rows locked / transaction open → baaki users ruk jaayenge, Mongo 60s limit, pool connections phanse.

Output

Step 1	[TRANSACTION]	stock reserve (stock - 3) + order create (status: PENDING_PAYMENT) + payment row (status: CREATED, idempotencyKey) → COMMIT (fast)
Step 2	[NO TX]	Payment gateway call / user UPI app mein pay karta hai
Step 3	[TRANSACTION]	Webhook aaya: payment SUCCESS → order PAID, payment SUCCESS → COMMIT Payment FAILED / 15 min timeout → order CANCELLED + stock wapas (+3) (timeout wala cleanup job / TTL-driven worker karega)

- Webhook do baar aa sakta hai → `gateway_payment_id` pe **unique index** + **UPDATE ... WHERE status = 'CREATED'** (idempotent). Part 19 mein detail.
- Transaction **chhota aur fast** rakho: sirf DB kaam, koi HTTP, email, file upload, ya **await sleep** andar nahi.
- Ye pattern (multi-step, compensating action) ko **Saga** kehte hain — microservices mein bhi yahi. Naam yaad rakho.

COMMON MISTAKES

- pg mein `pool.query('BEGIN')` — alag connections. Hamesha `pool.connect()` client.
- `client.release()` / `endSession()` bhoolna → connection leak.
- Mongo mein ek operation pe `{ session }` bhool gaye → wo transaction ke bahar chala, rollback nahi hoga. Silently galat.
- Prisma interactive transaction mein `prisma.x` use kar liya `tx.x` ki jagah.
- UPDATE 0 rows ko success maan lena (stock nahi tha, order ban gaya).
- Transaction ke andar HTTP call / email — lambi transaction, retry pe duplicate email.
- Local Mongo standalone pe transaction test — 'Transaction numbers are only allowed on a replica set member' error.
- Har cheez ko transaction mein daal dena — single-row/single-doc atomic updates ko transaction ki zaroorat nahi.

INTERVIEW QUESTION

Q: Place order flow mein transaction kaise use karoge? Payment gateway call kahan hogi?

Answer: Stock decrement (conditional: stock >= qty), order + items insert, aur payment record — ek transaction mein, taaki partial state na bane. SQL mein BEGIN/COMMIT ek hi pooled connection pe; Mongo mein `session.withTransaction` (replica set, har op mein session). Gateway call transaction ke bahar: pehle order PENDING_PAYMENT + stock reserve commit, phir gateway, phir webhook pe doosri chhoti transaction se PAID ya CANCELLED + stock release. Idempotency ke liye gateway payment id pe unique index. Mongo mein items embed karne se order write single-document atomic hai, transaction sirf cross-collection kaam ke liye.

MUST KNOW	GOOD TO KNOW	NOT NECESSARY RIGHT NOW
• ACID ek-ek line mein + example • BEGIN/COMMIT/ROLLBACK • Conditional update (stock >= qty) + rowCount check • pg: same client; mongo: session har op mein • External calls transaction ke bahar • Mongo single-doc atomicity + replica set requirement	• Isolation levels + defaults (PG RC, MySQL RR) • withTransaction retry behaviour • PG aborted state vs MySQL • Saga naam • Prisma tx / Sequelize transaction	• MVCC internals • WAL / redo log / journal internals • 2PC (two-phase commit) internals • Savepoints (rarely needed)

Part 12 — Memory Table

Kaam	SQL	MongoDB	Node
Start	BEGIN / START TRANSACTION	<code>session.startTransaction()</code>	pg: <code>client.query('BEGIN')</code> ; mysql2: <code>beginTransaction()</code>
Commit	COMMIT	<code>commitTransaction()</code>	<code>withTransaction auto</code>
Rollback	ROLLBACK	<code>abortTransaction()</code>	catch mein; <code>withTransaction throw</code> pe auto
Scope	Connection	Session	<code>pool.connect()</code> / <code>{ session }</code>
Atomic without tx	Single statement	Single document	Embed / conditional update
Requirement	—	Replica set	

Kaam	SQL	MongoDB	Node
Conflict	Lock wait / deadlock error	WriteConflict → retry	withTransaction retries
ORM	prisma.\$transaction / sequelize.transaction	mongoose session	tx.* / { transaction: t }

PDF 3 — Final Recap (Parts 9-12)

- 3 anomalies + 1NF/2NF/3NF ek line mein; denormalization consciously (snapshot vs copy).
- Mongo: access-pattern-first; embed vs reference rules; unbounded array anti-pattern; order + items design.
- Index kya, kyun fast, cost; queries ke liye index; ESR rule; leftmost prefix; unique index = race-proof.
- TTL (Mongo) vs cron cleanup (SQL); partial unique for soft delete; production index build (CONCURRENTLY, autoIndex off).
- EXPLAIN padhna teeno DBs mein; COLLSCAN/Seq Scan/ALL; SORT/filesort; docsExamined vs nReturned.
- Slow query log, timeouts, debug checklist.
- ACID; SQL + Mongo transactions; Node.js sahi pattern; external calls bahar; Saga.

Practice task (1 hour)

- Node script se 5 lakh fake orders insert karo (Postgres + Mongo). `GET /users/:id/orders` query ka EXPLAIN index ke bina aur ESR index ke saath compare karo — numbers note karo.
- Jaan-bujhkar galat index (`user_id, total, created_at`) banao, `total > 500 ORDER BY created_at` query ka SORT stage/filesort dekho.
- `placeOrder` dono DBs mein implement karo. Test: stock = 2, qty = 3 → 409 aur DB mein koi order nahi. Beech mein `throw` daal ke rollback verify karo.